

EAI-SIM: An Open-source Embodied AI Simulation Framework with Large Language Models

Guocai Liu¹, Tao Sun², Weihua Li^{1,3}, Xiaohui Li¹, Xin Liu¹, Jinqiang Cui^{1*}

Abstract—In the era of embodied intelligence, developing and testing various intelligent algorithms for robots, including unmanned aerial vehicles (UAVs) and manipulators, poses significant challenges. Researchers increasingly depend on simulation technologies to evaluate the performance of new algorithms across various conditions and applications. The advent of large language model (LLM) technology creates new demand for simulations of robotics tasks with the help of LLMs. This study introduces EALSIM, a simulation system capable of controlling unmanned aerial vehicles (UAVs) and manipulators based on user prompts. The modular framework acts as an extension of NVIDIA Isaac Sim, facilitating real-time simulation of UAVs and manipulators in photo-realistic environments. Thanks to the modular implementation and user-friendly Graphical User Interface (GUI), it can be seamlessly integrated with the widely used PX4-Autopilot and ROS2 system. To showcase its performance, a variety of control Application Programming Interfaces (APIs) are implemented within EALSIM. A range of tasks can be achieved by combining different APIs. Supplementary material, including code and documentation for this framework, is provided to benefit the community. The source code can be found at <https://github.com/PengICS/eai.sim>, and a demo video can be found at <https://youtu.be/YcNkwo.PNnQ>.

I. INTRODUCTION

The rapid progress of LLMs, such as GPT-3.5/4 [2], Llama, have exhibited powerful capabilities and strong potential for various applications, e.g. text generation, inference, and code generation. Unlike text-only applications, robotics systems necessitate a profound comprehension of real-world and environmental conditions, and the capability to execute safe actions. It is a prerequisite for AI-powered robotic models to have a basic understanding of the real world, e.g. physics constraints, and allow it to interact with users, e.g. transform and execute commands in a manner that aligns with the dynamic physical constraints in the real world.

This paper aims to develop a framework EAI-SIM that combines robotics with LLMs [2] in a robotics simulation environment (NVIDIA® Isaac Sim [1]), providing a powerful platform for simulating common robotics that is robust to environment conditions and easy to deploy, as shown in

*This work was supported by The Major Key Project of PCL Department of Broadband Communication, China Postdoctoral Science Foundation(2023M731829), and the Talent Program of Guangdong Province (2021QN02Z107). Corresponding Author: Jinqiang Cui

¹Guocai Liu, Xiaohui Li, Xin Liu, Jinqiang Cui are with Peng Cheng Laboratory, Shenzhen, Guangdong 518055, China. {liugc, liux, lixh01, cuijq}@pcl.ac.cn

²Tao Sun is with Shenzhen Institute of Artificial Intelligence and Robotics for Society, Shenzhen, Guangdong, China. tsun9tic@gmail.com

³Weihua Li is with Beihang University, Beijing, China. wehuali89@gmail.com

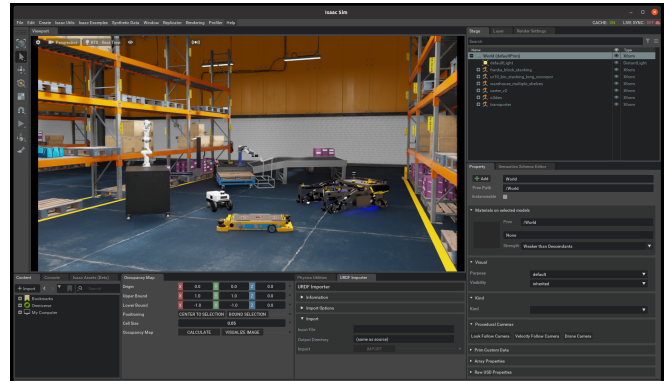


Fig. 1. EALSIM Development Interface

Fig.1. The diverse nature of robotics, encompassing various platforms, scenarios, and sensors, corresponds to a variety of libraries and APIs. Integrating LLMs with specific platforms may necessitate extensive fine-tuning. Therefore, A simple high-level function library was generated for the LLM to handle [7]. The library can then be seamlessly integrated with the actual APIs of the chosen platforms in the backend. Consequently, we empower LLMs to interpret user intentions from natural dialogue and translate it into a sequence of high-level function calls. Additionally, we present a couple of guidelines for prompts design to help LLMs tackle robotics tasks effectively.

Our main contributions are as follows:

- A generic pipeline of deploying LLMs on robotics tasks in Isaac Sim simulation environment is introduced. The pipeline enables a couple of prompting techniques, e.g. zero-shot natural language dialogue, code prompting, and role assignment. A demonstration is given to show a user can address tasks only by a simple prompt.
- We introduce a collaborative open-source platform based on Isaac Sim, EALSIM, where researchers in the community can share their prompting strategies to distinguish which leads to good or bad outcomes, specifically robotic actions in different conditions. Since prompt engineering mostly relies on empirical trial-and-error, it is better to have researchers contribute knowledge as a community.
- We provide two example simulations, including a px4-based multirotor and a Franka manipulator. These ample environments for drone navigation and manipulation aim to be a platform and community for enthusiasts of AI and robotics to investigate how LLMs can be

integrated into various robotics scenarios.

The remainder of this paper is organized as follows. Section II gives an overview of related work in robotics with respect to LLMs. Section III details the methodology and system architecture. Section IV presents the simulation results, and Section V concludes the paper.

II. RELATED WORK

With the rapid development of AI and robotic techniques, the capabilities of LLMs in the domain of task planning for robotics have been investigated in many works. Various simulation platforms with different features and properties have been released.

Webots [13], a free and open-source platform for robot simulators, was released in 2004. It integrates an experimental environment, a variety of models of robots, sensors, and actuators for robotic control into one platform for robot algorithm development. It not only provides various robot models but enables the modification of models physically and graphically according to specific needs, including shape, color texture, sensors, and hardware. However, compared to other platforms, e.g. Isaac Sim, the environmental scene generated by Webots is monotonous, and lacking in realism.

The General Robot Intelligence Development (GRID) [14] was presented as a platform for robot applications, which provided a new paradigm of simulators for robot techniques. It encompasses a variety of features and capabilities - mimicking the environment with physical constraints and enabling robots to interact with the real-world environment and execute operations based on a series of basic actions according to the physical characteristics of different types of robots. With the aid of the foundation model, robots can understand the real world with common knowledge which empowers the interaction of robots and users via natural language. The system can be installed on a local machine easily and runs with Docker. However, based on the paradigm, reusing source code and modification are not friendly. The simulator proposed in the paper provides a more friendly and complete open-source platform for robot algorithm development and verification.

Other platforms for robotic development include PromtCraft-Robotics [15]. Its development is based on the engine of AirSim which contains many existing experimental scenes for robot applications such as landing UAVs in offshore drilling platforms and Using UAVs for inspections in abandoned industrial facilities. The PromtCraft-Robotics only provides a simple interface that allows users to interact with LLM [10], e.g. ChatGPT-3.5 by prompt. The basic operations or actions API are inherited from AirSim which limits its application in more complex tasks and lacks robot models. a lot of work integrating natural language interaction into robotic systems have been done, for exam[e] language-based human-robot interaction [3][4], and visual-language manipulation [5][6].

III. METHOD

A. System Architecture

To verify the effectiveness of the pipeline, a simulation framework is developed for fast prototyping. The framework comprises four distinct layers as shown in Fig. 2, facilitating comprehensive control of unmanned air vehicles by the Language Model with direct interaction in the NVIDIA Isaac Sim environment.

The first layer encompasses the Isaac Sim Environment, which can import USD files using Python scripts. The unmanned air system is described as a standalone USD file model and installed as an Isaac Sim extension with an interactive graphical user interface(GUI). The second layer implements the sensor suites and communication functions. Four types of sensors are modeled, including a barometer, magnetometer, Inertial Measurement Unit (IMU), and Global Positioning System (GPS). An Application Programming Interface (API) is also provided, featuring callback functions for seamless access to system states and data generated by each onboard sensor. The third layer operates as a standalone running process, incorporating communication with the LLMs and the execution of MAVLink or ROS messages. This layer additionally offers a higher backend API to exert control over UAV vehicles. The final layer is the LLM layer, which translates user-issued natural language commands into executable function codes. Typical LLMs, including ChatGPT and PengCheng Mind [11], are accessible in this layer.

B. Isaac Sim

Isaac Sim is a simulation platform developed by NVIDIA as part of the NVIDIA Isaac robotics platform. It serves as a powerful simulation environment for training, testing, and validating robotic algorithms and systems. The platform is designed to accelerate the development of robotics applications by providing a realistic and physics-based simulation environment. It offers a high-quality RTX rendering engine capable of operating independently from its PhysX-based physics engine, which serves as the foundation for many 3D video game titles. Additionally, it adheres to the Universal Scene Descriptor (USD) standard established by Pixar, ensuring compatibility with various 3D modeling tools like Blender, Maya, and 3DS Max for representing extensive and intricate 3D environments.

Based on the powerful simulation capabilities of Isaac Sim, we constructed a simulated environment of a family apartment, including static items such as beds, sofas, cabinets, and tables. Additionally, we integrated drones and Franka robotic arms into this simulated environment. The simulated environment of the family apartment is ultimately saved in Universal Scene Descriptor (USD) file format, which can be found in the "env" file directory of our source code. When running the source code, the USD(Universal Scene Descriptor) file will be imported to load our simulation environment. While the environment construction was based on the Isaac Sim desktop software, the functionality code

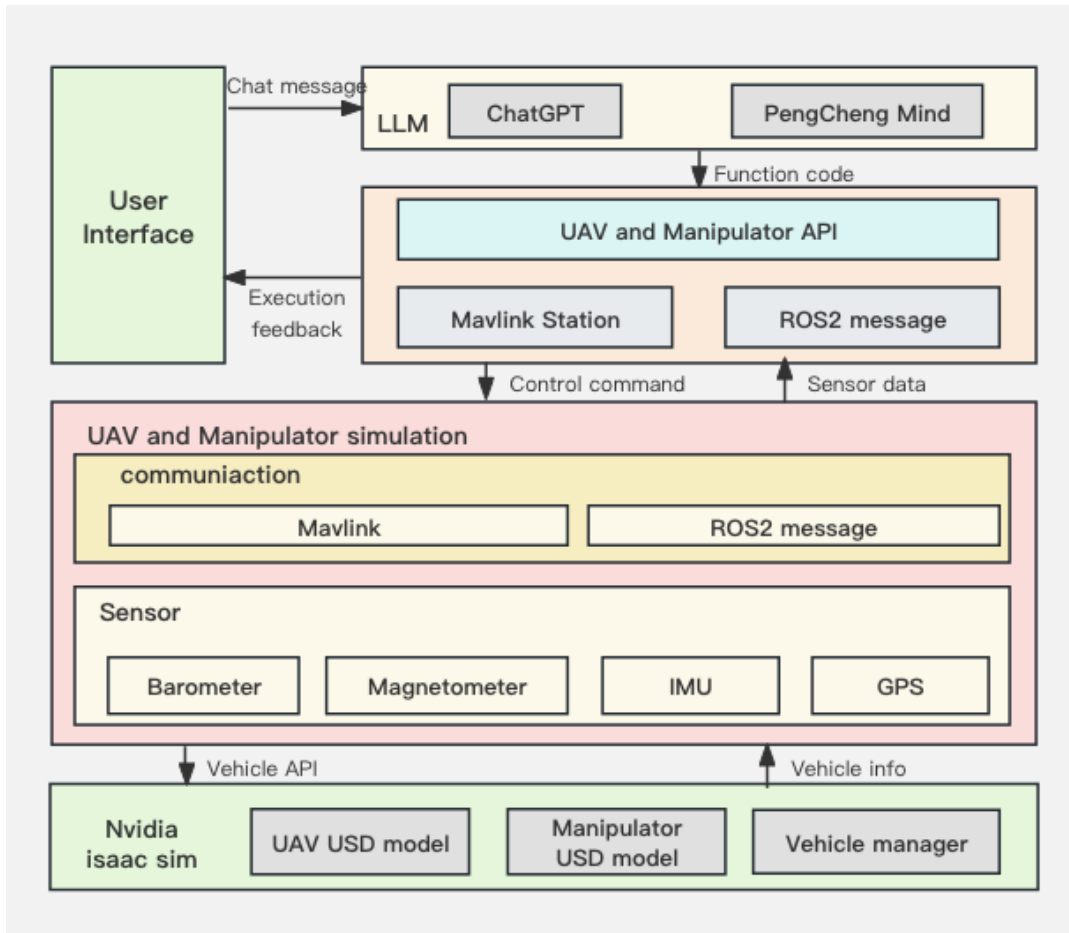


Fig. 2. The Architecture of EALSIM with LLM

primarily utilized the Isaac Sim Core API, which includes functions such as environment startup, loading, and executing robotic arm actions through API calls. Despite Isaac Sim providing powerful functionalities, it lacked robot management capabilities. Therefore, we developed a vehicle management module within the simulation framework for vehicle state management, including heartbeat detection, sensor status checks, and multi-threaded scheduling for multiple drones. Additionally, based on the underlying robot API of Isaac Sim, we developed a series of higher-level APIs for specific action completions. To run our environment, we utilized an Nvidia RTX 3070 graphics card, 32GB of RAM, and an Intel Core i7 processor.

C. Manipulator and UAVs

Our simulation platform is primarily tailored for robotic arms and drones, with the Franka robotic arm and a PX4-based drone platform as key components. Both the Franka robotic arm and the PX4 drone platform have been meticulously modeled using the Isaac Sim simulation platform, and control APIs have been implemented for each. Through the combined functionalities of these APIs, the robotic arm and drone can effectively carry out specified tasks. The USD files containing information about the relevant environments have

been made publicly accessible. The drone model utilized is referenced from the implementation by Pegasus simulation [12]. PX4, employed as the flight control software for the drone platform, is an open-source and versatile autopilot system. It is designed to govern various aerial vehicles, encompassing fixed-wing aircraft, multi-rotor copters, helicopters, and VTOL (Vertical Takeoff and Landing) vehicles.

D. Large Language Models

To achieve the control of unmanned systems through conversational interaction for task completion, we have incorporated different LLMs, including ChatGPT [2] and PengCheng Mind [11]. This helps us translate natural language into machine code that the unmanned system can execute. To implement these controls, we have written the functionalities and codes of the unmanned system's API into the prompts file, specifying each function and its corresponding action to LLM.

Our use of the LLM is facilitated through API calls, where individual commands are sent to the LLM like ChatGPT, which in turn returns the corresponding machine code [8] [9]. Ultimately, our simulation environment executes this machine code to fulfill the specified tasks. To enable this process, we need to provide pre-defined prompts to the

LLM. For the unmanned aerial vehicles (UAVs) and robotic arms, we have designed different prompts, which can be found in our open-source code repository. In the prompt design, our first step is to instruct ChatGPT to act as a robot assistant, tasked with helping us accomplish robot tasks. The second step involves specifying that only the execution code combinations provided by us should be returned by GPT. Next, we inform ChatGPT about the developed APIs, along with their corresponding functions and parameters. Lastly, we offer one or two demonstration examples for ChatGPT to learn from. Prompt engineering is primarily an empirical process, and while we have provided simple examples, we hope to enrich the robot-related prompts over time. Our prompts are as Fig.3.

You are an assistant helping me with the EALSIM simulator for drones.

1. When I request a task, your role is to provide Python code necessary to accomplish it using Isaac Sim. Following that, you should offer an explanation of the code's functionality.
2. You are restricted to using only the functions I have supplied to you. Any other speculative functions are prohibited. However, you may utilize basic Python functions from libraries like math and numpy.
3. Here are some functions you can use to command the drone.
aw.takeoff() - takes off the drone.
aw.arm() - make sure the drone ready to fly
aw.land() - lands the drone.
aw.back() - the drone go back to initial place
aw.get_position() - returns the current position of the drone as a list of 3 floats corresponding to XYZ coordinates.
aw.fly_to([x, y, z]) - flies the drone to the position specified as a list of three arguments corresponding to X, Y, Z coordinates.
aw.set_yaw(yaw) - sets the yaw of the drone to the specified value in degrees.
aw.get_yaw() - returns the current yaw of the drone in degrees.

A few useful things: Instead of utilizing moveToPositionAsync() or moveToZAsync(), you should utilize the function fly_to() which has been provided to you. If you have any uncertainties or require clarification on any aspect, feel free to ask me by explicitly stating "Question". Here's a scenario to illustrate how you can ask clarification questions. In terms of axis conventions, Forward indicates movement along the positive X-axis. Left indicates movement along the positive Y-axis. Up indicates movement along the positive Z-axis.

Fig. 3. LLM prompts

In addition to ChatGPT, our framework also integrates

PengCheng Mind, allowing us to control the robots in our simulation environment to complete tasks. PengCheng-Mind[11] 200B is a large-scale model based on the Transformer architecture, featuring a self-regressive language model with 201 billion parameters. The entire development process of the model is carried out on the fully autonomous, secure, and controllable domestic hardware and software platform provided by the hub node of the China National Computing Center in Peng Cheng Laboratory (Pengcheng Cloud Brain II). The training stage is implemented using the MindSpore framework, ensuring long-term stable multi-dimensional distributed parallel training on a large-scale cluster.

IV. SIMULATION RESULTS

A. PX4 simulation and control

In the simulation environment, we have implemented control for both single and multiple PX4[13] instances. For the single-instance control, we've achieved PX4's take-off, landing, and flight control in six directions: forward, backward, left, right, up, and down. These flight tasks can be accomplished solely through commands in a chat-based manner. The PX4 unmanned aerial vehicle operates within the Isaac Sim simulation software, where the control process and the simulation process of the drone are separate entities communicating through the MAVLink protocol.

For the multi-vehicle communication, each PX4 simulated drone corresponds to a unique port. By sending MAVLink protocol UDP messages to different ports, we can achieve communication among multiple drones. We abstracted the control algorithms of the drones into control interfaces. By combining calls to these interfaces, we achieve the control of unmanned drones to accomplish specific tasks. We informed ChatGPT about each interface and its corresponding functionality. The table below illustrates the control interfaces and their corresponding prompts. Ultimately, through conversing with ChatGPT, we guide the drones to perform specific tasks. ChatGPT, in turn, combines these APIs to accomplish the tasks we need. The APIs and functionality are as follows in Table I.

TABLE I
THE APIs AND FUNCTION OF UAVS

arm()	be ready to take off
takeoff()	make arm take off
fly_to(x,y,z)	fly to the x,y,z position
back()	fly back to the initial place
get_position()	get the drone's position

B. Single Vehicle Simulation

We have achieved control for a single drone (Fig.4.), through interactive conversation with the LLM. The conversation with the LLM is shown in Fig.5. The process demonstrates the LLM's ability to generate feasible code according to the user input. We requested the drone to take



Fig. 4. Single vehicle model

off, fly to the left, land at the original place, and perform other tasks. After executing the provided code, PX4 in the Isaac Sim simulation software completed the relevant tasks.

```
User: we have a drone named aw, please arm.
LLM: aw.arm()
User: please take off()
LLM: aw.takeoff()
User: please fly up 10 meter
LLM: position = aw.get_position()
      aw.fly_to(position.x, position.y, position.z+10)
```

Fig. 5. LLM controls a single UAV

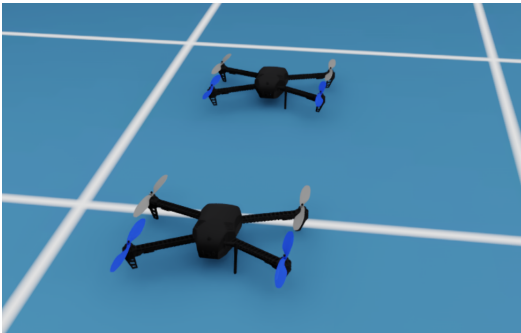


Fig. 6. Multi-vehicle model

C. Multi Vehicles Simulation

In addition to the single-drone control, we have also implemented a multi-drone collaborative control. The multi-drone model is shown in Fig.6. The flight control of each drone is implemented as a multi-threaded asynchronous process, enabling the simultaneous takeoff and execution of tasks by multiple drones, as well as queuing and waiting for tasks to be executed by multiple drones. All of these control operations can be completed through conversational interaction with the LLM. Communication among multiple drones is also facilitated through the MAVLink protocol, with each drone having a unique corresponding port number. Communication with different drones is achieved by sending UDP messages to different ports. The conversation with LLM in Fig.7 illustrates the control of two drones, with one drone

flying upward and the other flying to the left. The flying process is depicted in Fig.8.

```
User: we have two drones named aw1 and aw2,
      please arm.
LLM: aw1.arm()
      aw2.arm()
User: please let two drones take off
LLM: aw1.takeoff()
      aw2.takeoff()
User: please aw1 fly left 10 meter and aw2 fly
      up 10 meter
LLM: position1 = aw1.get_position()
      position2 = aw2.get_position()
      aw1.fly_to(position1.x, position1.y + 10,
                  position1.z)
      aw2.fly_to(position1.x, position1.y, position1.z + 10)
```

Fig. 7. LLM controls multiple UAV

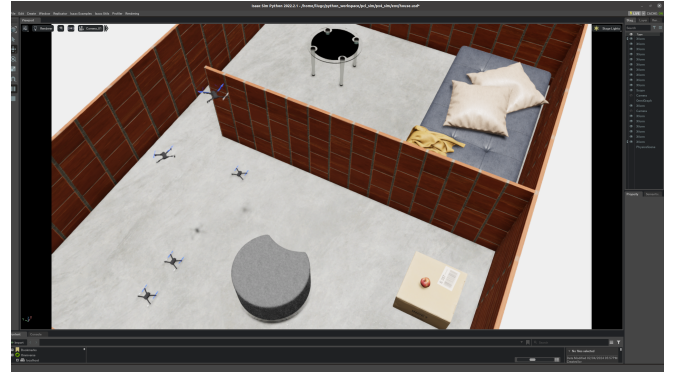


Fig. 8. Multi-vehicle flying different direction

D. Franka manipulator simulation and control

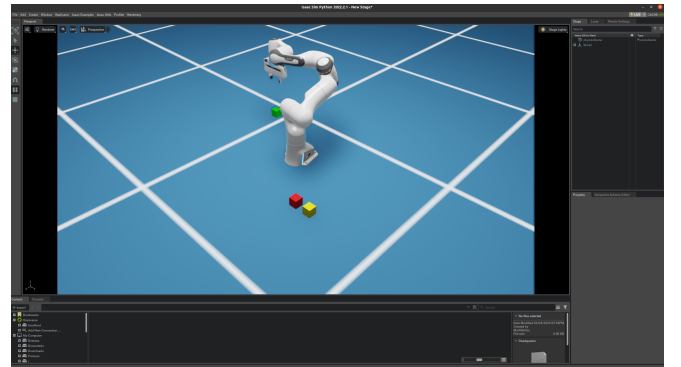


Fig. 9. Franka manipulator

In addition, we have also implemented simulation and control for a robotic arm. Unlike drones, communication for the robotic arm is achieved through socket messages. When running the Franka robotic arm simulation, we simultaneously start a co-routine to run a socket server. The

socket client operates in another independent process, which can also engage in conversation with the LLM and receive executable code in return. In our simulation environment, we have added three colored blocks (yellow, green, and red). Utilizing the Franka robotic arm, we have defined tasks to grasp specific color blocks. We informed the LLM about the APIs and functionalities required for grasping. The LLM successfully provided executable code, completing the task of grasping the three color blocks. The diagram in Fig.9 illustrates the process of the Franka robotic arm grasping the blocks.

V. CONCLUSIONS AND FUTURE WORK

We have presented a simulation framework designed to harness LLMs' potential in various robotics applications, involving the creation and implementation of a library of APIs for robot control. The simulation framework facilitates the testing, and validation of the generated code through various methods. We demonstrated the versatility of the framework across different tasks, including multi-drone operation and manipulation. The simulation framework is open-sourced to benefit the community. Our future work include adding more robotic platforms, incorporating sensor feedback in the simulator for simulation of multi-modal large models.

REFERENCES

- [1] NVIDIA Isaac Sim 2022, Accessed 10-Feb-2024. url:<https://docs.omniverse.nvidia.com/isaacsim/latest/index.html>
- [2] OpenAI. ChatGPT. Accessed: 2024-02-10. url:<https://openai.com/blog/chatgpt/>
- [3] Arthur Buckner, Luis Figueredo, Sami Haddadin, Ashish Kapoor, Shuang Ma, Sai Vemprala, and Rogerio Bonatti, "LATTE: LAngeage Trajectory TransformEr," 2023 IEEE International Conference on Robotics and Automation (ICRA), London, United Kingdom, 2023, pp. 7287-7294, doi: 10.1109/ICRA48891.2023.10161068.
- [4] Arthur Buckner, Luis Figueredo, Sami Haddadin, Ashish Kapoor, Shuang Ma, and Rogerio Bonatti. "Reshaping Robot Trajectories Using Natural Language Commands: A Study of Multi-Modal Data Alignment Using Transformers." 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (2022): 978-984.
- [5] Mohit Shridhar, Lucas Manuelli, and Dieter Fox. "Cliport: What and where pathways for robotic manipulation." In: Proceedings of the 5th Conference on Robot Learning. 2022 .
- [6] Yunfan Jiang, Agrim Gupta, Zichen Zhang, Guanzhi Wang, Yongqiang Dou, Yanjun Chen, Li Fei-Fei, Anima Anandkumar, Yuke Zhu, and Linxi Fan. "Vima: General robot manipulation with multimodal prompts." In: International Conference on Machine Learning(2023) .
- [7] Wenlong Huang, Pieter Abbeel, Deepak Pathak, and Igor Mordatch. "Language models as zero-shot planners: Extracting ctionable knowledge for embodied agents." In: International Conference on Machine Learning(2022)
- [8] J. Liang et al., "Code as Policies: Language Model Programs for Embodied Control," 2023 IEEE International Conference on Robotics and Automation (ICRA), London, United Kingdom, 2023, pp. 9493-9500, doi: 10.1109/ICRA48891.2023.10160591.
- [9] Singh, I., Blukis, V., Mousavian, A. et al. PROG PROMPT: program generation for situated robot task planning using large language models. Auton Robot 47, 999–1012 (2023).
- [10] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, Keerthana Gopalakrishnan, Karol Hausman, Alex Herzog, et al. "Do as I can, not as I say: Grounding language in robotic affordances." In: Conference on Robot Learning (CoRL) (2022).
- [11] PCL. Peng Cheng Mind. Accessed: 2023-02-08. 2023. url:<https://openi.pcl.ac.cn/PengChengMind/PengCheng.Mind>.
- [12] Marcelo Jacinto, João Pinto, Jay Patrikar. "Pegasus Simulator: An Isaac Sim Framework for Multiple Aerial Vehicles Simulation." In: arXiv preprint arXiv::2307.05263 (2023).
- [13] Michel, O."Webots: Professional Mobile Robot Simulation, <http://www.ars-journal.com/International-Journal-of-Advanced-Robotic-Systems/Volume-1/39-42.pdf>", Journal of Advanced Robotics Systems, Vol.1, No.1, pp.39-42,2004.
- [14] Sai Vemprala and Shuhang Chen and Abhinav Shukla and Dinesh Narayanan and Ashish Kapoor."GRID: A Platform for General Robot Intelligence Development", In: arXiv::2310.00887(2023)
- [15] Vemprala, Sai and Bonatti, Rogerio and Buckner, Arthur and Kapoor, Ashish."ChatGPT for Robotics: Design Principles and Model Abilities", In: arXiv::2306.17582(2023)